UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Bug Reporter

Assignement 1

Autori: Cimpan Alex-Ciprian, Ilco Doina-Cezara, Marcu Andrei
Grupa: 30233

Facultatea de Automatica
si Calculatoare

Martie 2026

Cuprins

1 Introducere **2**

2 Tech stack **2**

3 Arhitectură **2**

4 Modelul Entităților **4**
4.1 Diagrama Entităților 4
4.2 Enum-ul BugStatus 4
4.3 Relațiile dintre entități 4

5 Securitate **5**

6 Testare **5**
6.1 Teste de Repository 5
6.2 Teste de Service 6

7 Baza de Date **6**
7.1 Configurare 6
7.2 Tabele generate automat 6

1 Introducere

Bug Tracker este o aplicație web care permite utilizatorilor să raporteze, să urmărească și să gestioneze bug-uri software. Utilizatorii pot crea rapoarte de bug-uri cu tag-uri, pot comenta bug-urile existente și pot efectua operații CRUD complete pe toate entitățile.

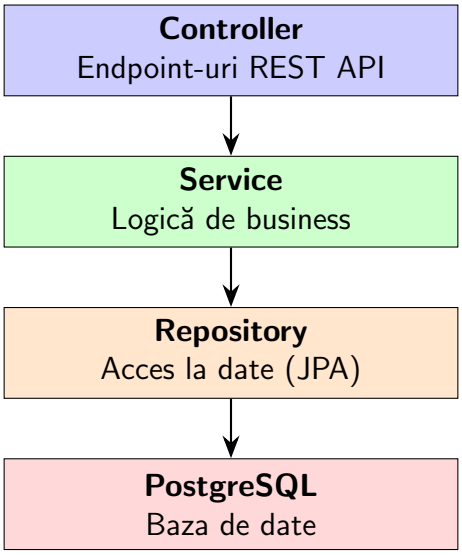
2 Tech stack

Tehnologie	**Versiune**	**Scop**
Java	17	Limbaj de programare
Spring Boot	4.0.3	Framework principal
Spring Data JPA	–	Acces la date / ORM
Hibernate	7.2.4	Implementare JPA
Spring Security	–	Criptarea parolelor (BCrypt)
PostgreSQL	18	Bază de date relațională
Lombok	–	Reducerea codului boilerplate
JUnit 5	–	Framework de testare unitară
Mockito	–	Framework de mock-uri pentru teste
Maven	–	Build și gestionarea dependențelor

Tabela 1: Tehnologiile utilizate în proiect

3 Arhitectură

Aplicația urmează o **Arhitectură pe Straturi** (3 niveluri), care separă responsabilitățile în straturi distincte:



- **Controller** – Gestionează cererile și răspunsurile HTTP. Adnotat cu `@RestController`.
- **Service** – Conține logica de business. Adnotat cu `@Service`.
- **Repository** – Interfețe care extind `JpaRepository` pentru operațiile cu baza de date.
- **Entity** – Entități JPA mapate pe tabelele din PostgreSQL folosind `@Entity`.
- **Config** – Configurarea securității (`SecurityConfig` pentru regulile HTTP, `PasswordEncoderConfig` pentru BCrypt).

4 Modelul Entităților

4.1 Diagrama Entităților

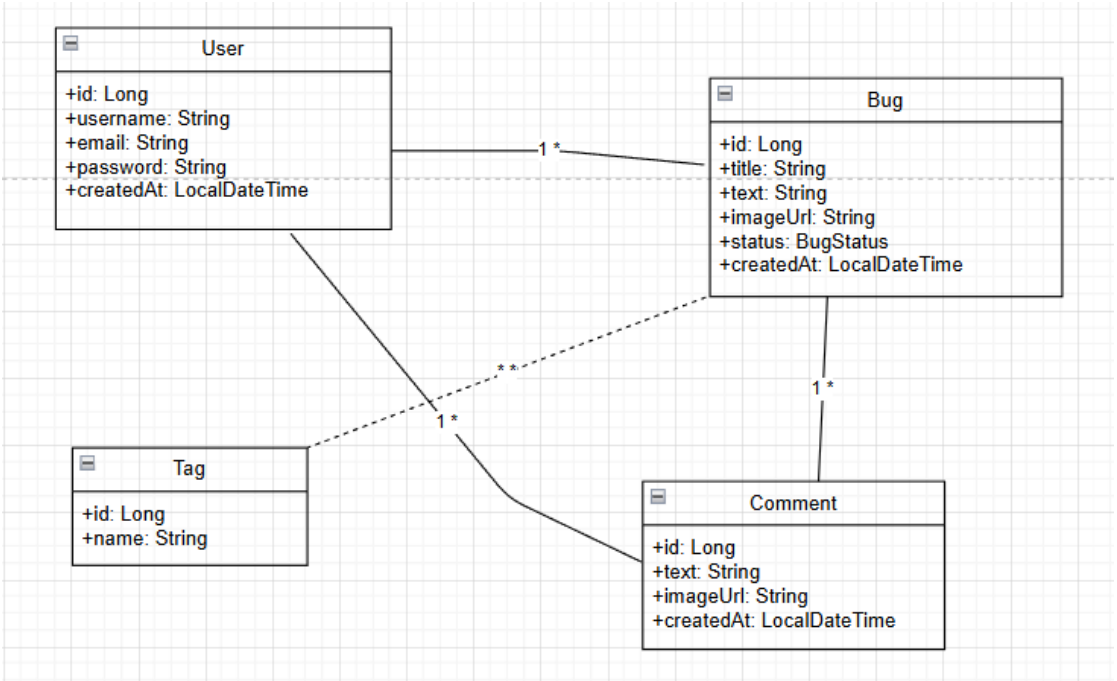


Figura 1: Entity Diagram

4.2 Enum-ul BugStatus

Bug-urile pot avea unul din următoarele statusuri:

```
public enum BugStatus {
    RECEIVED,       // Primit - cand utilizatorul posteaza bug-ul
    IN_PROGRESS,    // In progres
    SOLVED          // Rezolvat
}
```

4.3 Relațiile dintre entități

- **User** → **Bug**: Un utilizator poate fi autorul mai multor bug-uri (`one-to-many`).
- **User** → **Comment**: Un utilizator poate fi autorul mai multor comentarii (`one-to-many`).
- **Bug** → **Comment**: Un bug poate avea mai multe comentarii (`one-to-many`).

- **Bug ↔ Tag**: Un bug poate avea mai multe tag-uri și un tag poate fi asociat mai multor bug-uri (`many-to-many`), prin tabela de legătură `bug_tags`.

5 Securitate

Securitatea aplicației este configurată în două clase separate:

- **PasswordEncoderConfig** – Definește bean-ul `BCryptPasswordEncoder` pentru criptarea parolelor.
- **SecurityConfig** – Configurează regulile HTTP: dezactivează CSRF și permite accesul la toate endpoint-urile.

Fluxul de criptare a parolei:

1. Utilizatorul trimite parola în text clar prin request POST/PUT.
2. `UserService` apelează `passwordEncoder.encode()` înainte de salvare.
3. Parola este stocată în baza de date ca hash BCrypt (ex: `$2a$10$N9qo8uLO...`).
4. BCrypt este un algoritm unidirecțional – parola nu poate fi decriptată, doar verificată cu `passwordEncoder.matches()`.

6 Testare

Proiectul include două niveluri de testare:

6.1 Teste de Repository

Teste de integrare folosind `@DataJpaTest` cu o bază de date PostgreSQL dedicată pentru teste (`bugtracker_test`). Profilul de test este configurat în `application-test.properties` cu `ddl-auto=create` pentru o bază de date curată la fiecare rulare.

- `UserRepositoryTest` – salvare utilizator, căutare după username, căutare după email
- `BugRepositoryTest` – salvare bug cu autor, salvare bug fără autor (eșec așteptat)
- `CommentRepositoryTest` – salvare comentariu cu autor și bug, căutare după bugId
- `TagRepositoryTest` – constrângere de unicitate pe nume, căutare după nume

6.2 Teste de Service

Teste unitare folosind **Mockito** pentru a simula dependențele de repository.

- `UserServiceTest` – creare (cu criptarea parolei), obținere toți, obținere după ID, actualizare (cu re-criptarea parolei), ștergere, excepție la negăsit
- `BugServiceTest` – creare (status setat automat la RECEIVED), obținere toți, obținere după ID, actualizare, ștergere, excepție la negăsit
- `CommentServiceTest` – creare, obținere toți, obținere după ID, obținere după bugId, actualizare, ștergere, excepție la negăsit

7 Baza de Date

7.1 Configurare

Aplicația folosește două baze de date PostgreSQL:

- **bugtracker** – baza principală, folosită în dezvoltare și producție
- **bugtracker_test** – baza de test, cu `ddl-auto=create` (tabelele se recreează la fiecare rulare a testelor)

7.2 Tabele generate automat

Hibernate generează automat următoarele tabele pe baza entităților JPA:

- `users` – utilizatori (id, username, email, password, created_at)
- `bug` – bug-uri (id, title, text, image_url, status, created_at, author_id)
- `comment` – comentarii (id, text, image_url, created_at, author_id, bug_id)
- `tag` – tag-uri (id, name)
- `bug_tags` – tabela de legătură ManyToMany între bug și tag (bug_id, tag_id)